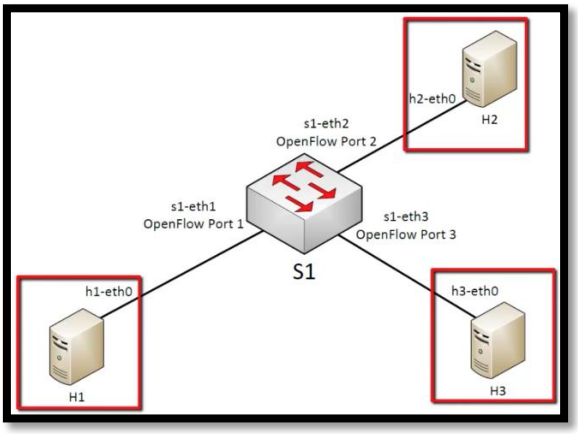
Managing flows in SDN manually

In this part of the lab, we will manage the flow entries on an Open vSwitch (OVS) manually using the ovs-ofctl command. Flow entries on an OpenFlow-capable switch control the behaviour of the packets. Normally these flows (or rules) are installed dynamically with an SDN controller.

**(see the ovs-ofctl man page also at openswitch.org).

Topology:



You'll use the network above, created using:

```
$ sudo mn --topo=single,3 --controller=none --mac
```

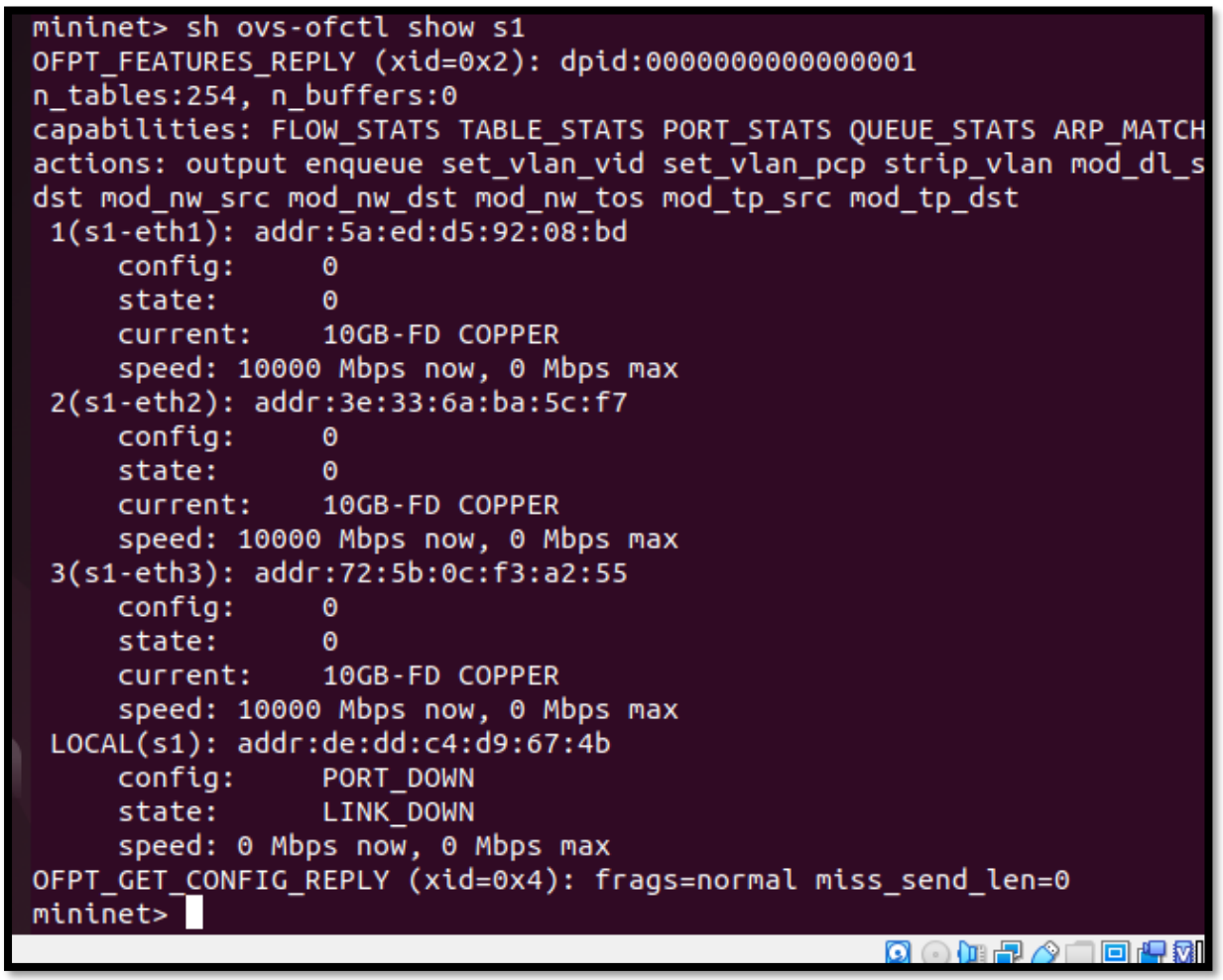
- --controller=none means that commands will be provided manually
- --mac means that MACs will be simplified

```
saim@saimvm:~/mininet/mininet$ sudo mn --topo=single,3 --controller=none --mac
[sudo] password for saim:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> sh ovs
```

Now, execute the dump and the net commands to check the topology.
The command to see how switch s1 ports map to OpenFlow Port (OFP) numbers is:

```
mininet> sh ovs-ofctl show s1
```



which basically means that OFP **1** refers to s1-eth1, OFP **2** to s1-eth2, and so on.

Flow Entries:

Before creating your first flow, use the following command to see if there are any flows already written on a switch.

```
sh ovs-ofctl dump-flows s1
```

Now use the following command to see if your ping is working. It should not work.

```
pingall
```

Let’s create the first flow entry:

```
mininet> sh ovs-ofctl add-flow s1 action=normal
```

normal means traditional switch behaviour, that is the classical switch forwarding operations. Let’s test with *pingall*.

After running the above command. You can use the ping commands to check the connectivity among hosts as,

H1 ping h2
H1 ping h3
H2 ping h3

Or simply use ***pingall*** command.

Now use the following command to see the flow output.

```
mininet> sh ovs-ofctl dump-flows s1
```

```
mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=183.955s, table=0, n_packets=30, n_bytes=2212, actions=NO
RMAL
```

Delete the entry (the flow) using:

```
mininet> sh ovs-ofctl del-flows s1
```

This command deletes all flows in s1. Now execute once again:

```
mininet> sh ovs-ofctl dump-flows s1
```

You’ll see that there are no more flows defined.

Now execute once again:

```
mininet> pingall
```

```
mininet> sh ovs-ofctl del-flows s1
mininet> sh ovs-ofctl dump-flows s1
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet>
```

USING LAYER-1 Data for flows

The flows can also be written based on traffic at switch physical ports. Here we will programme the switch so that everything that comes at openflow port 1 of s1 will be sent out to openflow port 2 of s1 and vice versa. You can look at the topology diagram above for switch 1 ports and traffic directions at ports 1 and 2 of the switch.

The required commands are:

mininet> sh ovs-ofctl add-flow s1 priority=500,in_port=1,actions=output:2

mininet> sh ovs-ofctl add-flow s1 priority=500,in_port=2,actions=output:1

The two instructions indicate the switch that what enters from port 1 must be forwarded to port2… and vice versa.

Now, ping the hosts as follows to see the connectivity.

mininet> h1 ping -c2 h2

mininet> h3 ping -c2 h2

```
mininet> sh ovs-ofctl add-flow s1 priority=500,in_port=1,actions=output:2
mininet> sh ovs-ofctl add-flow s1 priority=500,in_port=2,actions=output:1
mininet> h1 ping -c2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.428 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.059 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1011ms
rtt min/avg/max/mdev = 0.059/0.243/0.428/0.184 ms
mininet> h2 ping -c2 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.277 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.131 ms

--- 10.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1023ms
rtt min/avg/max/mdev = 0.131/0.204/0.277/0.073 ms
mininet> h3 ping -c2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1029ms
pipe 2
mininet>
```

Now execute once again:

mininet> sh ovs-ofctl dump-flows s1

```
 cookie=0x0, duration=296.696s, table=0, n_packets=6, n_bytes=476, priority=500
,in_port="s1-eth1" actions=output:"s1-eth2"
 cookie=0x0, duration=278.520s, table=0, n_packets=6, n_bytes=476, priority=500
,in_port="s1-eth2" actions=output:"s1-eth1"
```

Now, you can see the two newly created flows and the information about them. The priority value is important. If a packet enters a switch and there are various rules, only the one with a higher value is executed.

Let's add another flow:

mininet> sh ovs-ofctl add-flow s1 priority=32768,action=drop

This flow has a higher priority (32768 which corresponds to the default value; Priorities range between 0 and 65535.)

Run the above dump-flows command again to see the flow entries.

Let's now eliminate such the command below (it deletes the flow with all the default parameters

```
mininet> h1 ping -c2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.325 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.145 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.145/0.235/0.325/0.090 ms
mininet> h1 ping -c2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.067 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.130 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1028ms
rtt min/avg/max/mdev = 0.067/0.098/0.130/0.031 ms
mininet> h3 ping -c2 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable
From 10.0.0.3 icmp_seq=2 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1014ms
pipe 2
mininet>
```

Host 3 is still unreachable because we have not written any flow for host 3 connected to port 3 of switch s1. (you can try adding it)

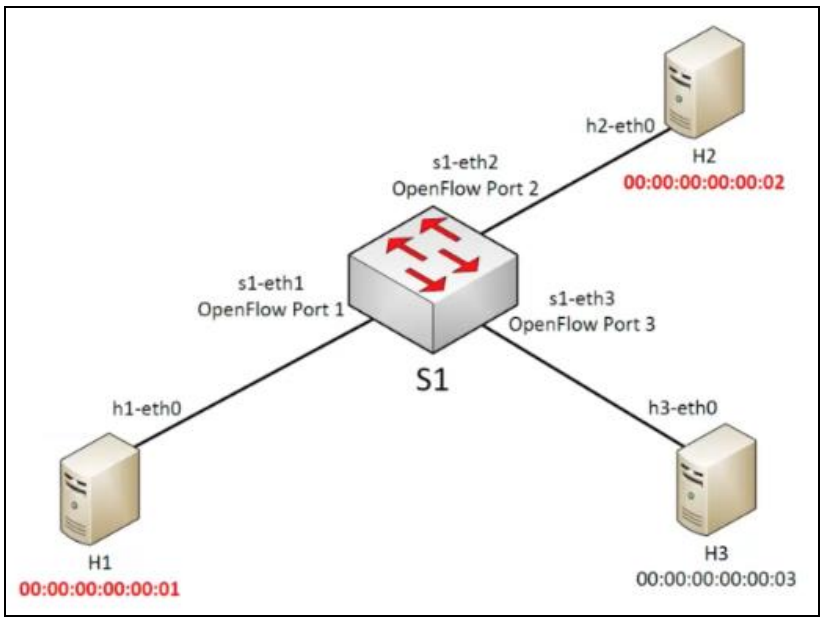
Now delete the flows using the following command and then try the ping command.

mininet> sh ovs-ofctl del-flows s1

```
mininet> sh ovs-ofctl del-flows s1
mininet> sh ovs-ofctl dump-flows s1
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet>
```

Using layer 2 data to control flows:

In this section, you will repeat the same operations but instead of using the port numbers, using the MAC addresses of the hosts. Since we execute mininet with the **--mac** option, the hosts will have simplified MAC addresses:



You can also use the ifconfig command to find the MAC addresses for each host.

Execute the command below:

mininet> sh ovs-ofctl add-flow s1
dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02,actions=output:2

mininet> sh ovs-ofctl add-flow s1
dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01,actions=output:1

Try *pingall* to see if it works.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet>
```

As you will see it doesn't work since ping works at IP level and the first thing it does is to use ARP to find out the MAC addresses of the different hosts. Our switch, with the rules it currently has, is filtering out ARP data traffic.

ARP is a broadcast protocol, so we need to add another rule:

mininet> sh ovs-ofctl add-flow s1 dl_type=0x806,nw_proto=1,action=flood

This rule adds a flow that “floods” all Ethernet frames of type 0x806 (ARP) to all the ports of the switch. nw_proto=1 indicates an “ARP request”. Since the replies in ARP are unicast, we already have the right flows set.

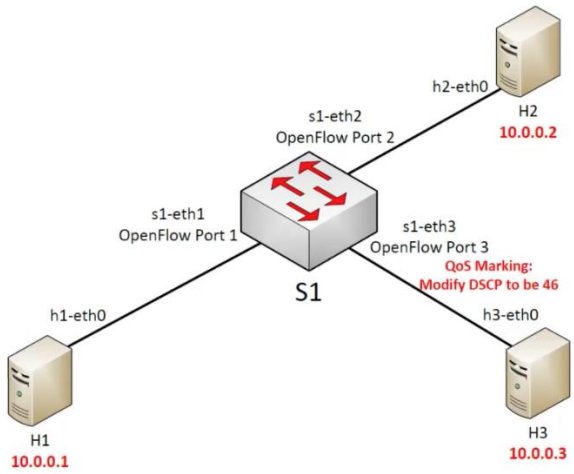
```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet> sh ovs-ofctl add-flow s1 dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:0
0:02,actions=output:2
mininet> sh ovs-ofctl add-flow s1 dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:0
0:01,actions=output:1
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet> sh ovs-ofctl add-flow s1 dl_type=0x806,nw_proto=1,action=flood
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 X
h3 -> X X
*** Results: 66% dropped (2/6 received)
mininet>
```

Now delete all flows again using del command from above.

```
mininet> sh ovs-ofctl del-flows s1
mininet> sh ovs-ofctl dump-flows s1
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X X
h3 -> X X
*** Results: 100% dropped (0/6 received)
mininet>
```

Using Layer 3 data to create flows:

In this section, we will use layer 3 (IP) information for the creation of flows.



All hosts will talk to one another, and we will give priority to data coming from h3 using DSCP, that is using DiffServ. In OpenFlow, there are many rules to modify packet contents.

Differentiated services or DiffServ is a computer networking architecture that specifies a simple and scalable mechanism for classifying and managing network traffic and providing quality of service (QoS) on modern IP networks. DiffServ can, for example, be used to provide low latency to critical network traffic such as voice or streaming media while providing simple best-effort service to non-critical services such as web traffic or file transfers.

First start by executing:

mininet> sh ovs-ofctl add-flow s1 priority=500,dl_type=0x800,nw_src=10.0.0.0/24,nw_dst=10.0.0.0/24,actions=normal

mininet> sh ovs-ofctl add-flow s1 priority=800,dl_type=0x800,nw_src=10.0.0.3,nw_dst=10.0.0.0/24,actions=mod_nw_tos:184,normal

We use the 184, to specify the 46 DSCP value in the IP TOS field, we must shift it to 2 bits on the left according to the meaning of the bits of the TOS field.

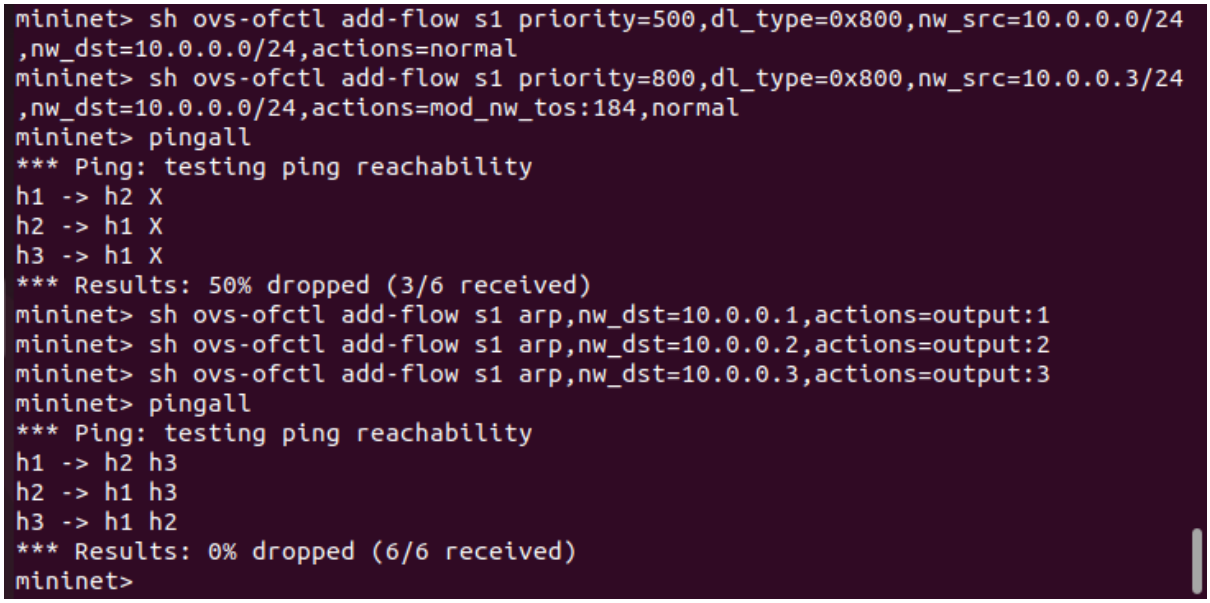
Now we must again enable ARP. We'll do it in a slightly different way:

mininet> sh ovs-ofctl add-flow s1 arp,nw_dst=10.0.0.1,actions=output:1

mininet> sh ovs-ofctl add-flow s1 arp,nw_dst=10.0.0.2,actions=output:2

mininet> sh ovs-ofctl add-flow s1 arp,nw_dst=10.0.0.3,actions=output:3

In this case, we are not using flooding, which can be a useful behaviour (imagine a 24 ports switch)



Now delete all flows using the command from previous sections.

Mininet> sh ovs-ofctl del-flow s1